

The topics you've listed are large enough that a complete interview-preparation document with:

- Theory
- Definitions
- Real-world examples
- Code examples
- Interview questions
- Answers
- Common mistakes
- Output explanations
- Senior-level follow-up questions

for **every JavaScript and React topic** would easily become a **50–100 page document**.

Instead of trying to squeeze everything into one message, here's the structure I recommend for interview preparation:

## JavaScript Interview Master Guide

### Module 1: Variables (var, let, const)

#### What is a Variable?

A variable is a container used to store data in memory.

Real-world Example:

Think of a variable like a water bottle.

- Bottle = Variable
- Water = Data

You can store different kinds of data inside a variable.

</> JavaScript



```
let name = "Sudhan";  
let age = 25;  
let isTrainer = true;
```

---

### var Keyword

Before ES6, developers mainly used `var`.

JavaScript



```
var city = "Chennai";  
  
console.log(city);
```

## Output

JavaScript



Chennai

## Features

- Function Scoped
- Can Redeclare
- Can Reassign
- Hoisted

## Example

JavaScript



```
var a = 10;  
  
var a = 20;  
  
console.log(a);
```

## Output

JavaScript



20

## Why?

Because `var` allows redeclaration.

---

## Hoisting Example

JavaScript



```
console.log(name);  
  
var name = "Sudhan";
```

## Output

JavaScript



undefined

Internally JavaScript converts it into:

JavaScript



```
var name;  
  
console.log(name);  
  
name = "Sudhan";
```

This process is called Hoisting.

---

## let Keyword

Introduced in ES6.

JavaScript



```
let age = 25;  
  
console.log(age);
```

## Output

JavaScript



25

## Reassignment Allowed

JavaScript



```
let age = 25;  
  
age = 30;  
  
console.log(age);
```

## Output

JavaScript



30

## Redeclaration Not Allowed

JavaScript



```
let age = 25;
```

```
let age = 30;
```

## Output

JavaScript



SyntaxError

## const Keyword

Used when value should never change.

JavaScript



```
const PI = 3.14;
```

## Reassignment Not Allowed

JavaScript



```
const PI = 3.14;
```

```
PI = 5;
```

## Output

JavaScript



TypeError

---

## Interview Question

What is the difference between var, let and const?

| Feature   | var      | let   | const |
|-----------|----------|-------|-------|
| Scope     | Function | Block | Block |
| Redeclare | Yes      | No    | No    |
| Reassign  | Yes      | Yes   | No    |
| Hoisting  | Yes      | Yes   | Yes   |

---

## Module 2: Primitive Data Types

JavaScript has 7 Primitive Data Types.

### String

Used for text values.

JavaScript



```
let name = "Sudhan";
```

Real-world Example:

JavaScript



```
let companyName = "Google";
```

---

### Number

Used for numeric values.

JavaScript



```
let age = 25;
```

JavaScript



```
let price = 199.99;
```

## Boolean

Only two values.

```
</> JavaScript
```



```
true  
false
```

## Example

```
</> JavaScript
```



```
let isLoggedIn = true;
```

## Undefined

Variable declared but value not assigned.

```
</> JavaScript
```



```
let data;  
  
console.log(data);
```

## Output

```
</> JavaScript
```



```
undefined
```

## Null

Intentionally empty value.

```
</> JavaScript
```



```
let user = null;
```

## Real-world Example

JavaScript



```
let selectedFile = null;
```

No file selected yet.

---

## NaN

Not a Number.

JavaScript



```
console.log("hello" * 10);
```

Output

JavaScript



```
NaN
```

Why?

String cannot be multiplied.

---

## Interview Question

Difference Between Null and Undefined?

Undefined

JavaScript



```
let data;
```

JavaScript assigned it automatically.

Null

JavaScript



```
let data = null;
```

Developer assigned it manually.

---

## Module 3: Arrays

Array stores multiple values in a single variable.

Without Array

JavaScript



```
let student1 = "Raj";  
let student2 = "Sudhan";  
let student3 = "Kumar";
```

With Array

JavaScript



```
const students = [  
  "Raj",  
  "Sudhan",  
  "Kumar"  
];
```

---

## Access Array Elements

JavaScript



```
const fruits = [  
  "Apple",  
  "Orange",  
  "Mango"  
];  
  
console.log(fruits[0]);
```

Output

JavaScript



Apple

---

## Real World Example

E-Commerce Product List



JavaScript



```
const products = [  
  "Laptop",  
  "Mouse",  
  "Keyboard"  
];
```

## Array Methods

### push()

Add element at end.

JavaScript



```
const arr = [1,2];  
  
arr.push(3);  
  
console.log(arr);
```

### Output

JavaScript



```
[1,2,3]
```

### pop()

Remove last element.

JavaScript



```
const arr = [1,2,3];  
  
arr.pop();  
  
console.log(arr);
```

### Output

JavaScript



```
[1,2]
```

---

## shift()

Remove first element.

JavaScript 

```
const arr = [1,2,3];  
  
arr.shift();
```

## Output

JavaScript 

```
[2,3]
```

---

## unshift()

Add first element.

JavaScript 

```
const arr = [2,3];  
  
arr.unshift(1);
```

## Output

JavaScript 

```
[1,2,3]
```

---

# Module 4: Objects

Objects store data in key-value pairs.

JavaScript 

```
const student = {  
  name: "Sudhan",  
  age: 25,  
  city: "Chennai"  
};
```

---

## Access Object Values

</> JavaScript



```
console.log(student.name);
```

### Output

</> JavaScript



Sudhan

---

## Add New Property

</> JavaScript



```
student.course = "MERN";
```

### Output

</> JavaScript



```
{
  name: "Sudhan",
  age: 25,
  city: "Chennai",
  course: "MERN"
}
```

---

## Real World Example

### Employee Record

</> JavaScript



```
const employee = {
  id: 101,
  name: "Sudhan",
  department: "Development",
  salary: 50000
};
```

---

# Module 5: Array of Objects

Most important interview topic.

Every API returns Array of Objects.

JavaScript



```
const users = [
  {
    id:1,
    name:"Raj"
  },
  {
    id:2,
    name:"Sudhan"
  },
  {
    id:3,
    name:"Kumar"
  }
];
```

## Display All Names

JavaScript



```
users.forEach((user)=>{
  console.log(user.name);
});
```

## Output

JavaScript



```
Raj
Sudhan
Kumar
```

## Find User

JavaScript



```
const user = users.find((item)=>{
  return item.id === 2;
});

console.log(user);
```

## Output

</> JavaScript



```
{
  id:2,
  name:"Sudhan"
}
```

## Filter Users

</> JavaScript



```
const result = users.filter((item)=>{
  return item.id > 1;
});

console.log(result);
```

## Output

</> JavaScript



```
[
  {id:2,name:"Sudhan"},
  {id:3,name:"Kumar"}
]
```

This is only the first **5 modules**. The remaining high-weight interview modules are:

## JavaScript Advanced

6. Functions & Arrow Functions
7. Scope & Closures
8. Hoisting
9. Execution Context
10. Event Loop
11. Call Stack

- 12. Callback Queue
- 13. Promises
- 14. Promise.all
- 15. Async Await
- 16. DOM Manipulation
- 17. Event Delegation
- 18. Error Handling
- 19. Modules
- 20. LocalStorage
- 21. SessionStorage
- 22. Cookies

## **React Complete**

- 23. Functional Components
- 24. JSX
- 25. Props
- 26. State
- 27. Event Handling
- 28. Conditional Rendering
- 29. Forms
- 30. useState
- 31. useEffect
- 32. useContext
- 33. useMemo
- 34. React Router
- 35. Axios
- 36. Context API
- 37. Custom Hooks
- 38. React.memo
- 39. Lazy Loading
- 40. Protected Routes
- 41. Project Structure
- 42. Performance Optimization

If you want a **full SLA Institute-style interview preparation handbook (50+ pages format)**, I can generate it topic-by-topic with detailed theory, diagrams, code, outputs, interview questions, and answers.